

THEOPHILUS BIJU JOHN

Houston, TX | +1 445-309-4373 | theophilusjohn@gmail.com
linkedin.com/in/theophilus-b-john | theophilusjohn.com

SUMMARY

Software engineer with 3+ years spanning backend development and production infrastructure. Built enterprise banking systems in Java and SQL at TCS; now administer mixed Linux/Windows servers and ship full-stack and systems projects to production.

EDUCATION

University of Houston – Clear Lake

Houston, TX

Master of Science, Computer Science | Jul 2025 – May 2027 (Expected)

Karunya Institute of Technology and Sciences

Coimbatore, India

Bachelor of Technology, Computer Science and Engineering | 2018 – 2022

EXPERIENCE

Research Assistant, University of Houston – Clear Lake

Houston, TX

May 2026 – Present

- Administer a **4-server mixed Ubuntu Linux and Windows Server environment** supporting **100+ students and faculty** across authentication, web hosting, file services, and databases.
- Designed and deployed an **automated backup system using hardlinked rsync snapshots** with semester-based retention, providing daily restore points at a fraction of full-copy storage cost; hardened remote access with **chrooted, per-user SFTP** and group-based rules.
- Diagnosed and resolved production **IIS authorization failures and SQL Server TLS binding errors**; relocated web content and database files onto dedicated volumes to isolate OS and application storage.
- Maintain a **PHP / SQL Server accreditation reporting application** used for ABET assessment, resolving course-mapping and session-state defects.

Assistant Systems Engineer, Tata Consultancy Services

New Delhi, India

Jul 2022 – Jul 2025

- Developed and maintained backend components for **enterprise core banking software** on the **TCS BaNCS** platform using **Java and SQL**, delivering and supporting features across scheduled release cycles.
- Contributed to a development team resolving **30+ production defects annually**, diagnosing and fixing issues in live banking environments to improve transaction reliability.
- Partnered with QA, business analysts, and client teams across time zones through the full Agile cycle — sprint planning, code review, and UAT support.

PROJECTS

Philoi — Real-Time Collaborative Document Editor | philoi.net

Next.js, TypeScript, Yjs, PostgreSQL

- Built and deployed a production Google Docs-style editor supporting **concurrent multi-user editing with live presence cursors**, using **Yjs CRDTs over WebSockets** (Hocuspocus) for conflict-free merging; verified document convergence across **10 simultaneous clients**.
- Enforced document permissions at **three independent server-side layers** — REST authorization, WebSocket authentication, and transaction-level rejection of writes from read-only clients — after identifying that client-side read-only mode was bypassable via ProseMirror keyboard shortcuts.
- Shipped to production on a self-managed Linux host behind Caddy with managed PostgreSQL, **Cloudflare R2 object storage**, and **GitHub Actions CI/CD**; tuned to **10–12 ms keystroke latency** and ~340 ms cold document load.

Homonoia — Raft Consensus Implementation & Visualizer | homonoia.dev

TypeScript, React, Vitest

- Implemented the **Raft consensus algorithm** — leader election, log replication, and commit safety — as a **pure, I/O-free state machine**, driven by a deterministic simulated network supporting message loss, reordering, duplication, and network partitions.
- Asserted **all five Raft safety properties after every tick across 1,000 randomized seeds × 3,000 ticks** (~2M messages, 8.4 leader changes per seed) with seeded replay for reproducible failures; **128 tests**, full sweep in ~10s.
- Demonstrated that random fuzzing **cannot** surface the Figure 8 commit violation — with the current-term restriction removed, **5,000 seeds over 15M ticks produced zero detections** — and covered it instead with a deterministic scripted scenario failing at a fixed tick.

Enargeia — Browser-Native LLM Inference Engine | enargeia.dev

WGSL, WebGPU, TypeScript

- Implemented transformer inference **entirely in WGSL compute shaders with no ML framework**, running Qwen2.5-0.5B-Instruct fully client-side with **int4 block-wise quantization**.
- Sustained **43.1 tok/s decode and 1,109 tok/s prefill** at **335 MiB resident weights**; cut cold load to **14.6s** via CDN and object-storage optimization.

TECHNICAL SKILLS

Languages: Java, Python, TypeScript / JavaScript, C, C++, SQL, WGSL, Bash, PowerShell

Web & Data: Next.js, React, Node.js, Express, Prisma, Yjs / CRDTs, WebSockets, WebGPU, PHP, IIS, PostgreSQL, SQL Server, MySQL

Infrastructure: Linux (Ubuntu), Windows Server, Active Directory, LDAP / LDAPS, SSSD, NFS, OpenSSH, RAID, backup & recovery

Cloud & DevOps: GitHub Actions CI/CD, Caddy, Cloudflare R2 / Pages, DigitalOcean, Railway, Playwright, Vitest, Git